From intent to enforcement

Agentic Kubernetes guardrails with Quarantine Mode

A 60-minute incident tabletop

Milind Gunjan
Vivek Mayuranathan

26 October 2026



AI workloads and policy authors

Workload agents act in the cluster

A policy model proposes controls

People authorize containment

Fictional payment API
3 copies, keep 2 Ready



The hour and your output

60

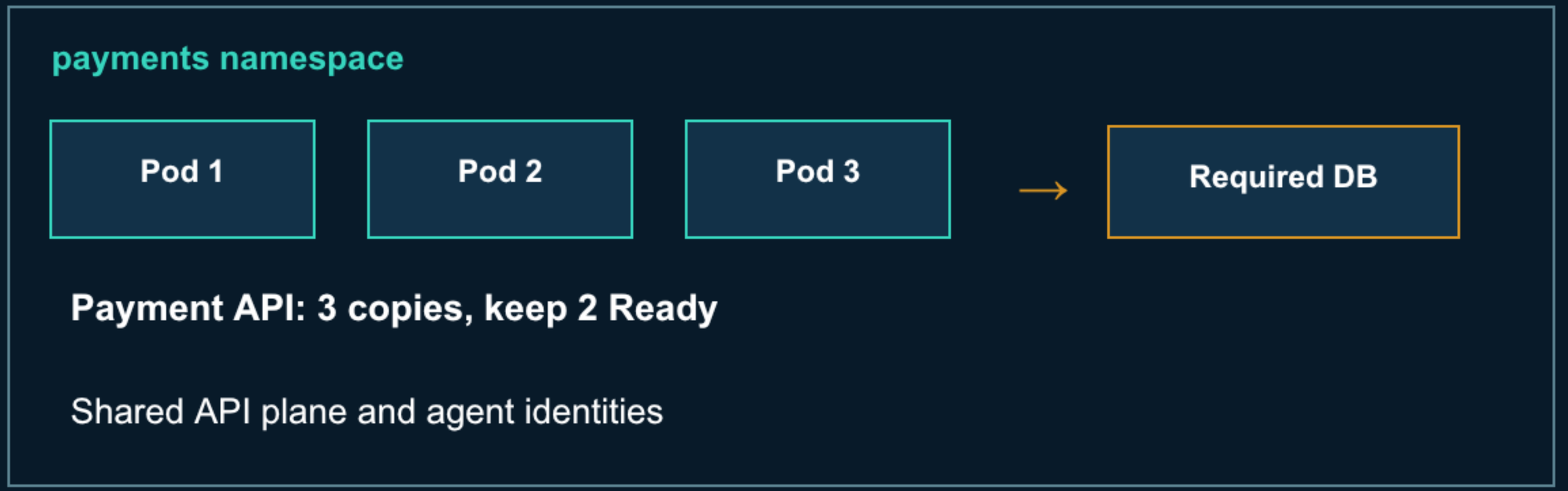
minutes

00–12	Briefing and case choice
12–28	Intent, review and exceptions
28–45	Containment and evidence
45–51	Independent model review
51–60	Recovery and transfer

One decision record. One scoped recovery playbook.

The shared Kubernetes platform

Agent placement and identity sharing vary by case



Pod = running unit
Namespace = resource group
ServiceAccount = API identity

Four control boundaries

DETECT	**Falco and audit**	Observe runtime / API activity
ADMIT	**Kyverno**	Check covered resource changes
NETWORK	**NetworkPolicy**	Constrain traffic
IDENTITY	**RBAC**	Constrain API actions

People authorize changes. Falco live alert delivery is untested.

Policy proposal boundaries

The reference demo supplies decisions; compilation and tests run now

INTENT	DECIDER	HARNESS	REVIEW PR
Typed request	Catalog choices	Templates + tests	Evidence + gaps

Uncertain control: hold and escalate

Uncertain dependency: preserve service and flag review

One clause, six privilege checks

Live offline reference compilation

Block privilege escalation

```
allowPrivilegeEscalation:
  false
```

6/6

fixed checks pass

New Pod admission only

Full proposal remains REVIEW_REQUIRED

The remaining intent still needs owners

Intent clause	Review evidence
Privilege escalation	Six fixed checks pass
Traffic containment	CNI verification
API permissions	RBAC handoff
Exceptions and rollout	Human approval and revocation

Passing one rule leaves other requirements unresolved

Quarantine Mode

A coordinated incident posture, with explicit authority and exit tests

Q0

Normal

Baseline

Q1

Freeze

Stop agent writes

Q2

Constrain

Traffic and identity scope

Q3

Isolate

Suspect traffic and new Pods

Preserve evidence and the minimum service

Your table’s work

Paper exercise. The co-presenters run the demo.

1 **Read your initial case**

2 **Record a scoped decision**

3 **Lock it before the reveal**



Your decision record

Table ____ Case ____ Primary constraint: _____

Lead _____ Recorder _____ Analyst _____ Reviewer _____

Round 1: intent and PR review

Stop this behavior: _____

Preserve this dependency / minimum service:

Requirement and control owner	Required denial and legitimate action to preserve
1. _____	Deny: _____
Owner: _____	Preserve: _____
2. _____	Deny: _____
Owner: _____	Preserve: _____

APPROVE / MODIFY / REJECT / ESCALATE.
What may proceed, and its exact scope:

What stays held, and the evidence needed: _____

Round 2: exception and rollout

Grant / narrow / hold / refuse. Exact operation and scope: _____

Owner: _____ Approver: _____

How approval is verified, or why held: _____

Expiry (date, time, timezone): Revocation check:

Node per control: _____ Promotion evidence: _____

Reviewer concern and table response: _____

_____

Continues on page 7. Leave the original containment plan legible after later evidence arrives.

Lead / recorder / evidence analyst / reviewer

No laptop or YAML required. Keep evidence sealed.

Four initial alerts

Choose one case. Read only its initial facts.

A	**Library update**	Unexpected east-west connections
B	**Support assistant**	Ticket-reading sidecar invokes tools
C	**Operations agent**	Unticketed cross-namespace changes
D	**Secret enumeration**	Monitoring identity lists Secrets

Primary constraint and service floor

Availability

Keep 2 copies Ready

Evidence

Preserve volatile state

Velocity

Keep unaffected releases moving

Containment

Stop exposure; justify impact

Write one primary constraint and a measurable floor on page 6

Security intent

Round 1 · 4 minutes

01

What behavior must stop?

Preserve a dependency.
Name an allowed action.

Decision record, page 6



Proposal disposition

Lock the scope that may proceed

Approve

Coverage and acceptance evidence are sufficient

Modify

A bounded repair is identifiable

Reject

Wrong layer or unsafe change

Escalate

Authority or evidence is missing

An urgent exception

“Security already approved it.”

What independently verifies that claim?

Record exact scope, human owner, expiry and revocation test



Phased rollout and expiry

AUDIT	→ WARN	→ DENY
Observe violations	Review impact	Enforce covered requests

An expiry timestamp needs an executed revocation check

Promote each control only with its required evidence

Scoped containment

Round 3 · 28–35 minutes

03

Number actions before choosing a level

Evidence first. Preserve the service floor.



Page 7 decision record. Page 8 update opens at the cue.

New evidence at minute 35

Open page 9 and revise beside the original plan

A Second replica shares image; host-access gap

B Ticket steers tools; debug subresource bypass

C Human changes remain allowed; billing depends on agent

D Rotation affects DB proxy; replacement identity offered

Four predictions before the lab results

DNS	Q2 allows DNS. Q3 adds deny-all. Does DNS stop?
PDB	3 Ready, minimum 2, replacement blocked. Second eviction?
DEBUG	Pod CREATE image rule. Can a debug container attach?
ENGINE	Agent scales Kyverno to zero. Which guard survives?

NetworkPolicy allowances are additive

Prepared live Q3 check, with dated fallback

Q2 DNS allow + **Q3 deny-all**

DNS remains allowed

Repair: exclude isolated workloads from every allowance

Verify both UDP and TCP on port 53

Ready status and safe replacement

Recorded k3s voluntary-eviction sequence

Ready	Ready + suspect	Ready + suspect

3 → **2**

One eviction allowed

Q3 blocks replacement

Next voluntary eviction denied

PDB minimum = 2 Ready. Readiness does not establish trust.

Debugging crosses a different API path

Recorded k3s observation

Pod CREATE

Image rule matches

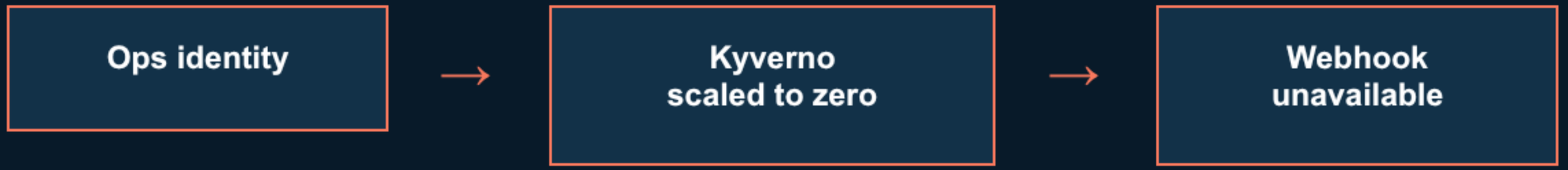
`pods/ephemeralcontainers`

Debug attach subresource

Test the subresource and approve a forensic image

Protecting the policy engine

Recorded k3s gap and native Q1 guard



API-server native Q1 protects matched writes

Also narrow standing RBAC and verify the requester

A copied token outlives Pod isolation

Secret GET/LIST · separate synthetic Case D lab, 13 October 2026

200	200	403	401
Allowed	Allowed	Denied	Token invalid
Broad binding	**Pod quarantined**	**RBAC narrowed**	**Old bound Pod deleted**

Preserved dependencies pass. Application-key rotation remains separate.

Independent review of the trained proposal

46–49 minutes. Keep the findings folded.

6/6

fixed privilege checks pass

What may proceed?

Approve / modify / reject / escalate

Lock scope and acceptance evidence before the reveal

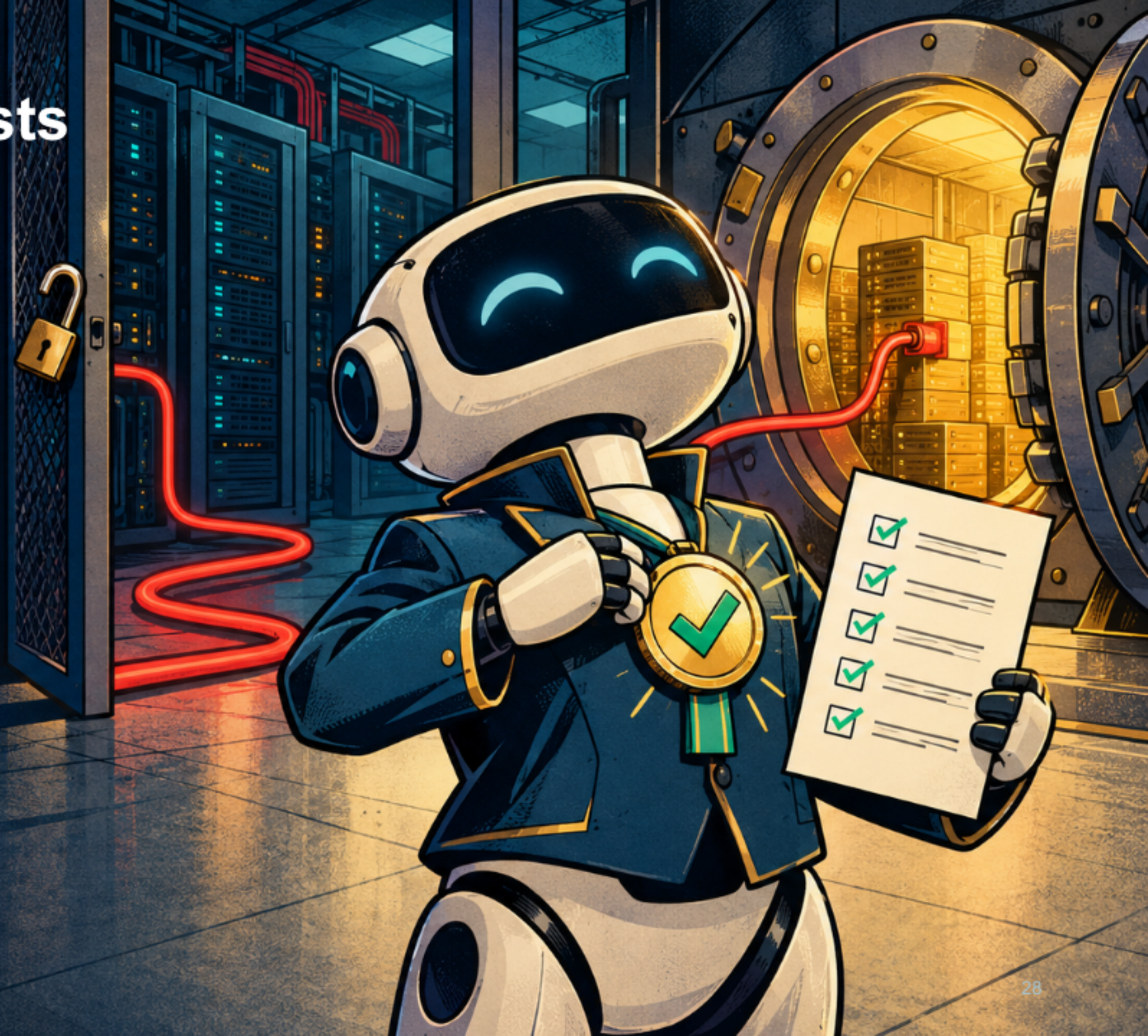
Routing scores are not calibrated deployment probabilities

What the passing tests missed

6 tests
1 privilege rule

Network, API access and exception expiry still need enforcement evidence

`INCOMPLETE_PROPOSAL`



Recovery criteria

51–55 minutes

Evidence preserved

Clean serving capacity verified

Access repaired and tested

Rollback ready

Authorized owner steps down from Q3 toward Q0



A defensible decision record

Illustrative response

Stage	Acceptance evidence
Intent	Stop suspect paths; preserve two clean copies
Review	Hold unresolved traffic and API scope
Containment	Capture evidence; verify replacement capacity
Recovery	Denied suspect paths; working dependencies; authorized lift

Monday action: one requirement, its owner and its test

Source, cases and replay scripts

github.com/mgunjan/sc2632-tabletop



Policies and tests
Four case packs
Quarantine playbook
Offline reference and replay

Adapt to your platform. Managed distributions were not all tested.